DECISIONLENS: TEAM STRATEGIC ROADMAP & ALIGNMENT

From Academic Prototype to B2B AI Decision Room · Engineering & Viva Playbook

Executive Summary: *We are transforming DecisionLens from an academic prototype with a 6-stage manual form into a high-value B2B platform:* ***The Collaborative AI Decision Room****. Instead of forcing busy executives to fill out step-by-step cognitive boxes, our new direction allows teams to conduct their meetings directly inside the app, captures silent anonymous dissent to kill meeting bias (HiPPO effect), and instantly outputs an audit-grade 1-page Architecture Decision Record (ADR) with closed-loop precedent memory. This document explains the exact 'Why', 'What', and 'How' so our entire team is aligned.*

1. Why Are We Changing? (The Flaws in Our Previous Direction)

Our backend architecture is exceptionally solid: we have 143 passing tests, an epistemic cognitive filter (Dr. Evelyn Vance), a gap sentry (Dr. Marcus Vance), and mathematical MAUT scoring (Prof. Caleb Thorne). **However, our product packaging had 3 fatal flaws:**

- **The 'Cognitive Obstacle Course' (High Friction):** Users were forced to manually click through 6 sequential tabs (Input → Structuring → Gaps → Frameworks → Synthesis → Roadmap). Busy tech leads and managers will not spend 15 minutes doing homework for a tool.
- **Single-Player Bottleneck:** Decision-making in engineering teams is never done alone. In our old model, only one person sat at their laptop entering text. The real stakeholders were excluded.
- **The 'HiPPO' Problem (Loud Boss Dominance):** In real 45-minute meetings, the senior lead speaks 80% of the time. Junior developers and quiet architects who know critical flaws are afraid to speak up aloud.

2. The New Direction: Collaborative Decision Room & ADR Engine

We are pivoting our user experience into a seamless **In-App Decision Room** that operates on 4 core superpowers:

Superpower	How It Works Technically	Why It Wins (The Value)
1. In-App Video & Silent Scribe	Embedded WebRTC video (Jitsi React SDK) + Private Side-Drawer on the right side of the screen.	**Kills HiPPO bias.** Team speaks on video, but quiet engineers privately and anonymously submit technical risks without office politics.
2. Trilingual Indian STT	AI4Bharat IndicWhisper + Groq Whisper-Large-v3. Native support for Punjabi, Hindi, and Indian English.	**No language barrier.** Code-switching in Indian tech teams (Bangalore, NCR, Chandigarh) is transcribed with 95%+ accuracy for &8377;0.
3. Dual Hub-and-Spoke RAG	Modular Vector Collections (Career, Legal, Software) + Private Document RAG for PRDs and contracts.	**Grounding without bloat.** Satisfies our teacher's academic demand for Universal RAG while keeping latency under 8 seconds.
4. Precedent Folder Memory	Semantic vector retrieval over past meetings in that folder, replacing naive LIMIT 5 FIFO.	**Prevents repeated mistakes.** If Meeting #2 had a database outage 5 months ago, Meeting #15 is warned automatically.

3. Concrete Technical Changes Required in Our Codebase

To make this new architecture live, here are the exact files and modules we are modifying. Notice that our existing 143 passing unit tests remain intact because our additions are modular adapters:

Component / Layer	Target File(s)	Action Required
Live Video Room	`frontend/components/LiveDecisionRoom.tsx`	Embed Jitsi Meet React SDK (70% width) with room link generation for multi-device joining.
Silent Scribe Drawer	`frontend/components/SilentScribeDrawer.tsx`	Right-side drawer (30% width) with inputs for anonymous technical risks, constraints, and criteria voting.
Express ADR Mode	`frontend/app/decisions/[id]/page.tsx`	Add 1-Click 'Generate 1-Page ADR Memo' button that runs Stages 2–5 asynchronously behind a 6-second progress bar.
Transcript Ingestion	`backend/app/services/file_parser.py`	Add parse_vtt_transcript() to strip WEBVTT timestamps from Google Meet and Zoom transcript exports.
Private Notes API	`backend/app/models.py` `backend/app/api/decisions.py`	Add DecisionParticipantNote table and POST /decisions/{id}/private-notes endpoint.
Folder Semantic RAG	`backend/app/services/folder_context.py`	Replace chronological LIMIT 5 with vector similarity search against past decisions in the same folder.

4. Team Division of Responsibilities (Who Builds What)

To execute this cleanly and without overlapping merge conflicts, here is the suggested division of labor across our team:

Role / Owner	Primary Ownership	Deliverables for This Sprint
Frontend Engineer 1 (UI & Live Room)	Live Decision Room & WebRTC Layout	1. Install @jitsi/react-sdk. 2. Build the split-screen Live Room page. 3. Connect the camera/mic controls.
Frontend Engineer 2 (Scribe & Express UX)	Silent Drawer & 1-Page ADR View	1. Build the Silent Scribe Drawer. 2. Build the 'Express Memo' 1-click summary card. 3. Hook up the PDF Export button.
Backend Engineer 1 (STT & Ingestion)	VTT Parser & Trilingual STT Gateway	1. Add VTT/TXT parser in file_parser.py. 2. Wire up AI4Bharat / Groq Whisper endpoint. 3. Create Participant Notes API.
Backend Engineer 2 (RAG & Folder Memory)	Folder Precedent Semantic RAG	1. Upgrade folder_context.py with vector search. 2. Connect Stage 3 gap sentry to domain RAG. 3. Maintain 100% pytest passing rate.

5. The Viva & Mentor Presentation Defense Playbook

When teachers, mentors, or external examiners question our project during reviews, every teammate must use these exact, unified defenses:

Examiner / Mentor Question	How Every Teammate Must Answer (The Unified Defense)
Q1: 'Why not just use ChatGPT Plus or Claude?'	*'ChatGPT has zero organizational memory and cannot track outcomes over 90 days. Furthermore, ChatGPT acts as an echo chamber for the loudest voice. DecisionLens has Precedent Chaining across meetings, silent anonymous risk capture, and mathematical MAUT scoring.'*
Q2: 'Your teacher asked for Universal RAG. Is it only for software?'	*'No, Sir. Our architecture is 100% Universal with a Hub-and-Spoke RAG model. We have modular vector collections for Career (O*NET), Legal (Common Paper), and Software (AWS Well-Architected). We use Software Architecture as our commercial spearhead because tech decisions carry multi-lakh financial penalties.'*
Q3: 'How does it handle Indian languages and Hinglish?'	*'We integrate AI4Bharat IndicWhisper (from IIT Madras), explicitly trained on Indian accents, code-switching, Punjabi, and Hindi. Furthermore, Stage 2 Dr. Evelyn Vance normalizes phonetic idioms into standardized engineering facts.'*
Q4: 'Why wouldn't an arrogant lead ignore your app?'	*'DecisionLens serves as an objective Architectural Sentry. It gives the lead an audit-grade liability shield (ADR) to justify the decision to the CTO, while allowing quiet junior engineers to privately submit fatal infrastructure risks.'*
Q5: 'What is your defensible moat?'	*'Folder Precedent Memory. When a company uses DecisionLens for 6 months, their past mistakes and post-mortems live inside our system. That institutional memory creates massive switching costs that no generic LLM can replicate.'*

Action Call for the Team: *Let us build with confidence. We have 143 passing tests, an unfair moat in Precedent Chaining, and a product that addresses real organizational pain. Review this document, pick your component, and let's make DecisionLens the standout project of our academic and professional careers.*